

Using Large Language Models in Critical Contexts

Techniques for robustness, reliability and rule-following,
applied to military Course-of-Action generation

Part I — The Company Period

Alexandre Hervé · A5 · Data & AI / UX Design

Thales SIX GTS France — Research & Development

Tutors: Aldo Maalouf (company) · Fatima Zahra KAGHAT (school)

Period: 02/02/26 – 31/07/26

Language: English · Confidentiality: Public

Contents

Acknowledgments	4
Glossary & Abbreviations	4
Abstract	6
1 Professional context of the internship	7
1.1 The economic and institutional sector	7
1.2 DD-RSE analysis of the sector	8
1.3 Positioning of the company	8
1.4 My position in the company	9
2 Internship presentation	9
2.1 Objectives and assigned projects	9
2.2 A comparative study: generative, deterministic, hybrid	10
2.3 Modelling the COA: doctrinal templates	10
2.4 Roadmap, methods and tools	11
2.5 Regulatory aspects, quality and risk management	12
2.6 The challenges identified at the outset	12
3 Progress of the internship	13
3.1 The missions	14
3.2 The two earlier builds and what they taught	14
3.3 How the architecture enforces its own principles	14
3.4 A walk through the pipeline	14
3.5 Personal achievements	15
3.6 Work in a doctrinal context	16
3.7 Probing robustness: the simulated adversary	16
3.8 Project ownership and self-imposed process	17
4 Internship analysis	17
4.1 Skills developed, acquired and missing	17
4.2 Difficulties and solutions	18
4.3 Successes, best practices, areas for improvement	19
4.4 Critical analysis, ethics, and DD-RSE commitments	19
4.5 Recommendations for continued work	20

4.6 Career plan	21
References	21

Acknowledgments

I would like to thank Claire Trefoux for welcoming me into her team, and the HexaForce project team for their reception. My special thanks go to Aldo Maalouf, my company tutor, for his guidance throughout this project, and to Fatima Zahra Kaghat, my ESILV tutor, for her support.

A note on sources and confidentiality. This report describes a research prototype built in a defence R&D setting. Where a detail would touch sensitive material it is kept at the level of principle rather than specifics, and the worked examples in the text are illustrative rather than drawn from any real operation. Public claims about the company and the Group are attributed to published sources; technical claims about the prototype reflect the state of the codebase at the end of the internship.

Glossary & Abbreviations

Term	Definition
COA	Course of Action: a candidate operational plan; the system’s primary output.
OPORD	Operation Order: the doctrinal document that tasks a unit; the pipeline’s primary input.
FRAGO	Fragmentary Order: an incremental update to a standing OPORD.
SITAC	<i>Situation Tactique</i> : the structured tactical picture (units, objectives, axes, phases) extracted from the order. Read-only downstream.
MAG	<i>Mode d’Action Générique</i> : a generic action mode; one of the eight doctrinal “lenses” the system treats as a single source of truth.
GOAP	Goal-Oriented Action Planning: the ontology paradigm used as the formal backbone for valid action decomposition.
EdG-T 2022	<i>École de Guerre — Terre, 2022</i> : the French Army tactical doctrine corpus the system must comply with.
HexaForce	A C2 software in development at Thales, targeted for the downstream integration of this project.
LLM	Large Language Model: used only where deterministic logic cannot reach, always behind a single client.
HITL	Human-In-The-Loop: mandatory operator review between pipeline stages; the system never auto-advances.
CPM	Critical Path Method: used to schedule the action network.
CP-SAT	Constraint-Programming SAT solver (OR-Tools): optional scheduler with a “first do no harm” fallback to a greedy heuristic.
RETEX	<i>Retour d’expérience</i> : after-action feedback, a real-time data source cited in the brief.
DD-RSE	Sustainable development and social responsibility: the responsible-AI and ethics frame the project sits within.
IHL	International Humanitarian Law: the legal frame (distinction, proportionality, precaution) that governs the use of force.

Term	Definition
Determinism-first	The project's core principle: deterministic computation wherever possible, the LLM only where necessary.

Abstract

This report describes an end-of-studies internship at Thales SIX GTS France, in a defence research-and-development setting, where I built a system that turns military operation orders into candidate Courses of Action (COA). The brief, set against the evolution of modern command, was to explore how artificial intelligence can assist a commander without reducing the work to fine-tuning a model on doctrinal data — the deliverable was a hybrid generator combining deterministic rules, optimisation, and AI, with an emphasis on system design, simulation, and multidimensional evaluation. The problem the work answers is the one the defence market gates everything else on: a planning aid is only useful if its output can be trusted, and a large language model, however fluent, produces wrong answers as confidently as right ones. My response was a *determinism-first* pipeline in which deterministic computation does the load-bearing work and the model is confined to a small, bounded surface. Every machine-touched field is checked by rules that do not trust it and reviewed by a mandatory human before anything proceeds.

The report is in two parts. Part I sets the company and sector context, the internship’s objectives and conduct, and an honest account of what was achieved and what proved difficult, including a measured efficiency weakness I report as a problem rather than a success. Part II, a separate scientific chapter, generalises the case study into a single question: how can LLMs be used in critical contexts so that their output is robust, reliable, and rule-following, both by how the model is built and by how the system around it is designed?

1 Professional context of the internship

The internship sits where defence decision-support meets applied AI, a field whose value is gated almost entirely by trust: a tool that cannot be verified cannot be fielded. This section sets the scene before the project itself: the company and the market it serves, the responsible-AI stakes that come with defence AI, how Thales positions itself, and where a single intern’s work fit inside a large R&D organisation. The thread running through all four subsections is one idea, returned to throughout the report: in this sector, trustworthiness is not a feature added to a capable product but the precondition for the product existing at all.

1.1 The economic and institutional sector

Thales SIX GTS France is a wholly-owned subsidiary of Thales SA, a large multinational now active in three high-technology growth markets: aerospace and space, defence and security, and cybersecurity and digital identity, employing tens of thousands of people worldwide. The subsidiary was created on 1 July 2011 from the merger of Thales Communications, the Group’s secure information and communication systems business for armed and security forces, with Thales Security Solutions & Services (initially under the name Thales Communications & Security, later renamed Thales SIX GTS France). Its name still carries the two activity domains it was built around: **SIX** for *Secure Information and Communication Systems* and **GTS** for *Ground Transportation Systems*. The SIX side covers secure communications, resilient networks, intelligence and surveillance, command-and-control, and information-systems security for defence and security customers. The GTS (rail and ground-transportation signalling) activity, by contrast, was divested: Thales completed the sale of its Ground Transportation Systems business to Hitachi Rail on 31 May 2024, so the rail business no longer sits inside the Group, and the “GTS” in the entity’s name is now largely a historical artefact rather than a live activity. The entity is headquartered in Gennevilliers and employs roughly 7,000 people across eight main sites in France. My internship sat squarely on the SIX side: the COA-generation project belongs to the defence, secure-information, and command-and-control domain, which is exactly the activity the subsidiary now concentrates on.

The market this places the work in is unusual, and understanding its shape explains most of the project’s design. It is not a consumer market. Customers are armed forces and state agencies; procurement cycles are long, often measured in years; and the buyer is frequently also the certifier, so a capability must satisfy not only the end user but a separate authority that decides whether it may be fielded at all. Demand is driven less by feature count than by whether a system can be trusted, integrated into existing doctrine and equipment, and audited after the fact. A capability that cannot be explained or verified is close to worthless here. A planning aid that occasionally produces a confident but wrong recommendation is not a faster planning aid; it is a liability, and in this domain a liability that can cost lives. That single economic fact, verification before raw capability, runs through everything that follows.

Competition in this market is therefore fought less on model size than on integration, sovereignty, and assurance. The relevant actors are the large defence primes and their software arms, specialist command-and-control vendors, and an increasing number of AI entrants, but the discriminating question among them is rarely “whose model scores higher on a benchmark”; it is “whose system can be certified, integrated, and trusted by an organisation that will be held responsible for the outcome”. A vendor that cannot answer that question does not lose on points; it is simply not in the conversation.

The wider context is that the character of military operations is changing, and the change is what makes decision-support AI a live research question rather than a convenience. The volume

of information a command post must absorb, the tempo at which decisions are demanded, the number of domains to coordinate, and the breadth of sources to fuse (geolocation, sensor feeds, after-action feedback or *RETEX*) have all grown faster than the staff available to process them. The classic staff process of generating several candidate courses of action, comparing them against the mission and the enemy, and recommending one to the commander, is labour-intensive and time-bounded in exactly the way automation is supposed to help. The standing motivation for a tool like the one I built is not to replace the commander’s judgement but to compress the staff-officer work of drafting and comparing options, so that scarce human judgement is spent where it actually matters. The internship is one small contribution to that long research programme.

1.2 DD-RSE analysis of the sector

The sustainable-development and social-responsibility (DD-RSE) stakes of defence AI are unusually concrete. Keeping a human in the loop and making machine reasoning auditable are not compliance checkboxes here; they are design constraints with ethical weight. When the output of a software tool can influence how force is applied, the usual responsible-AI concerns — transparency, accountability, the prevention of automation bias — stop being abstract governance language. They become decisions about where in the pipeline a human must sign off and which fields the machine is allowed to fill on its own.

This is also where defence AI meets the law of armed conflict. International humanitarian law turns on principles — distinction between combatants and non-combatants, proportionality of force, precaution in attack — that require situated human judgement and cannot be delegated to a statistical model. A tool that proposes a course of action is not making those judgements, but it is shaping the options a human will choose among, and a poorly designed tool can narrow or bias that choice without anyone noticing. The ethical requirement is therefore not only that a human signs off, but that the human signs off on a genuine understanding of what the machine produced and why, which is a much stronger condition.

Thales has taken a clear public position on this. The Group’s AI approach, branded “TrUE AI”, commits to AI that is **T**ransparent (the user can see the data behind a conclusion), **U**nderstandable (results can be explained and justified), and **E**thical (operation stays within legal and human-rights frameworks). The Group’s Digital Ethics Charter sets out, among its commitments on digital trust and responsibility, an explicit undertaking to keep humans in control of AI-based systems [1], [2]. For a defence context, the operative principle is simple to state and hard to honour: the machine proposes and a person disposes. An AI decision-support tool that quietly removed the operator from the loop would fail the ethical test regardless of how good its recommendations were. This is the frame the COA project inherits, and it maps almost one-to-one onto the architecture I built: a mandatory human-in-the-loop between stages, and visible provenance marking which fields the model produced. I return to the costs and residual risks of that frame in Section 4.4, with a candid assessment.

1.3 Positioning of the company

On its defence and secure-information side, Thales SIX GTS France differentiates on trustworthy, doctrine-aware, sovereign systems rather than on raw model capability. That posture is exactly what a determinism-first, auditable COA tool is built to express. A vendor whose pitch is “our model is bigger” competes on a dimension this customer base does not prioritise. A vendor whose pitch is “every output is explainable, doctrine-checked, reproducible, and overridable” is selling the thing the market requires before fielding any tool. Sovereignty matters too: a defence customer is wary of depending on a model or a service it does not control, which is one reason

the prototype keeps its model local rather than calling an external API.

That positioning is also why the Group invests in models adapted to its domains rather than only consuming general-purpose ones. A doctrine-aware language model developed inside Thales is one expression of that strategy: a model shaped toward military doctrinal language rather than left generic. It is cited in the internship brief as an inspiration, and it frames a question the project keeps returning to, namely how much of trustworthiness should be built into the model and how much should be enforced around it. The Part II chapter treats this as two complementary axes, trust by construction and trust by confinement; my work concentrated on the second, but the existence of the first shaped the framing, and it is a reminder that the wrapping-the-black-box stance is a deliberate choice rather than the only option.

1.4 My position in the company

I worked on the COA-generation system alone, reporting to a single company tutor, in a functional role best described as AI architecture and development within R&D. The current system is the third build of the project, after two earlier attempts of my own. Working inside an R&D organisation, even on a solo project, meant the work was not done in isolation from its context: the architectural direction was set in discussion with my tutor, and the operational plausibility of the output was checked at intervals with officers who could tell me when a plan was technically valid but tactically strange.

In practice, the role had two overlapping kinds of work. The first was research and planning: imagining the structure and architecture of the solution. The second was the actual development of the project. The two alternated: when a prototype reached its limit or became too bloated with additions and changes, a new planning and research period began, and once the skeleton of the next iteration was in place, development of the new generation started again.

I ran a disciplined, documentation-first process, described in Section 3.8, so that every change could be reproduced and reasoned about later, by me or by whoever picks the prototype up next.

2 Internship presentation

The brief was to explore how AI can assist command without reducing the work to fine-tuning a model on doctrinal text. The intended system was a *hybrid* generator of friendly Courses of Action, combining deterministic rules, optimisation, and AI, with the emphasis on system design, integration in a simulated environment, and a multidimensional evaluation of efficiency, explainability, and robustness. That framing, deliberately not “train a bigger model”, is what made the internship a systems-engineering project rather than a machine-learning one.

2.1 Objectives and assigned projects

Three objectives organised the work. The first was to build a prototype hybrid COA generator that supports decisions in near real time and stays consistent with French doctrine, combining generative AI with non-generative methods rather than relying on either alone. The second was to make the result *evaluable*: to measure efficiency, explainability, and robustness, ultimately including human tests with officers, rather than to demonstrate a single impressive run. The third was to explore how such a generator might integrate with larger systems, for example embedded or simulation environments such as HexaForce, without depending on intensive training over a doctrinal corpus.

A constraint cut across all three: *no fine-tuning*. The model was to be used as it came, adapted through structured prompting, post-processing, and rule-based validation rather than being re-trained. The rationale is worth spelling out because it shaped everything. Fine-tuning a defence-doctrine model is computationally intensive, data-hungry, and sensitive: it needs a large, clean, labelled corpus of doctrinal material, it bakes the training data into weights that are then hard to audit or correct, and in a classified domain the data itself is a liability. Avoiding it forces a different and, for this market, more interesting question: how far can a system be trusted without ever touching the model’s weights? (Thales has separately developed an array of models created explicitly with doctrinal compliance in mind; the point of this internship was to build the safety structure *around* such models rather than to add another fine-tuned one.)

It also means the prototype is portable across models: because nothing depends on a particular set of fine-tuned weights, a better or more domain-adapted base model can be dropped in without redoing the safety work.

The work I took on was a ground-up build. Rather than a feature list, it was framed as a set of properties the build had to *enforce structurally* rather than by convention: that only one module may call the model, for instance, and that the validator never trusts its upstream.

2.2 A comparative study: generative, deterministic, hybrid

The first phase of the work was a comparative study of how a COA could be generated at all, weighing a purely generative approach against a purely deterministic one, and arriving at the hybrid the rest of the project builds. Table 2 summarises the comparison that drove the decision.

Table 2: Approaches to COA generation considered in the design phase.

Approach	Strengths	Weaknesses
Generative AI (transformer LLM)	Fluent natural-language output; handles unstructured orders; flexible	Hallucinates; sensitive to phrasing; no constraint guarantee; hard to audit
Deterministic (rules, decision trees, optimisation)	Predictable; auditable; provably within constraints; reproducible	Brittle on free text; rigid; cannot read prose or phrase output naturally; extremely code-heavy
Hybrid (adopted)	Rules and optimisation carry the guarantees; the model handles language, intention and breadth; failures are caught by the rule layer	More moving parts; the boundary between model and rules must be designed carefully

The conclusion was that neither pure approach is acceptable in a critical context. A generative model alone cannot offer the assurances the market requires for fielding; a purely deterministic system cannot read an order written in prose or express a plan in the language an officer expects. The hybrid puts each where it is strong: the deterministic and optimisation components carry the load-bearing guarantees, and the model is confined to the parts where natural language is unavoidable, with its output always checked before it is trusted.

2.3 Modelling the COA: doctrinal templates

A second design-phase task was to define what a COA *is*, structurally, so that it can be generated and checked automatically. The system builds on the classic five-paragraph operations-order format, drawn from the *Mémento de tactique appliquée* of the École de Guerre-Terre [4] and

standard NATO/France order practice: situation, mission, execution, logistics, and command and signal. Casting the output as a fixed template is what makes both automatic generation and automatic checking possible, because every field has a defined place and meaning. The skeleton and a simplified illustrative example are shown below.

Five-paragraph order, skeleton.

- 1. Situation** — enemy; friendly forces; attachments/detachments; assumptions and uncertainties.
- 2. Mission** — action verb + who + what + where + when + why (the end state).
- 3. Execution** — general concept of the manoeuvre; main tasks of subordinate units; supporting tasks (fires, intelligence, logistics); coordination measures (boundaries, axes, phase lines).
- 4. Logistic support** — supply; medical; transport; other specific support.
- 5. Command and signal** — command posts; frequencies and call-signs; command handover if applicable.

Illustrative example (simplified, fictitious).

- 1. Situation.* A mechanised enemy company holds a defensive position around a named height; a friendly battlegroup of three infantry companies and an armoured squadron is assembled some kilometres to the south; an engineer element is attached for crossing support.
- 2. Mission.* The battlegroup attacks at a stated time to destroy the enemy on the height in order to open the brigade's advance to the north.
- 3. Execution.* Concept: surprise attack from the south-east, fixing the enemy by fire, then dismounted assault. First company fixes to the west; second company conducts the main assault; the armoured squadron provides direct support and acts as reserve.
- 4. Logistic support.* Full ammunition load plus one resupply; forward medical post at a designated location.
- 5. Command and signal.* Battlegroup command post at a stated location; tactical frequency assigned.

The example is fictitious and simplified; its purpose is only to show the shape the generator must fill. The point for the system is that this structure is the contract between the deterministic core, which decides the manoeuvre and the assignment of units, and the language layer, which renders the result into the order an officer reads.

2.4 Roadmap, methods and tools

The work followed the brief's six-month shape in three phases, summarised in Table 3.

Table 3: Six-month roadmap.

Months	Phase	Focus and deliverables
1–2	Analysis and design	Comparative study of approaches; modelling the COA templates; identifying the challenges (latency, real-time data, doctrinal and ethical compliance).
3–4	Development and integration	Deterministic and optimisation core; generative component via prompting and post-processing; operator interface; periodic checks with officers.
5–6	Evaluation and simulation	Multidimensional benchmarks; closed-loop simulation; ethical and human analysis; final report and demonstration.

In practice the first two phases consumed most of the time, and the third is where the honest gaps remain, as Section 4.3 records. I worked phase by phase against written pass/fail gates: a stage was not “done” because it ran, but because it met a criterion I had committed to in advance. That discipline, more than any single technical choice, kept the rebuild honest.

The toolset was deliberately conventional; Table 4 lists the main pieces and their role. The system is written in Python, with typed schemas enforcing the contract at every stage boundary so that a stage cannot hand the next a malformed artefact. The operator interface is built in Streamlit, chosen for fast iteration on a menu-driven, one-decision-per-screen design that never auto-advances. Model calls go to a locally-hosted LLM, ensuring confidentiality and reproducibility. Two references anchored the work: Sebastian Raschka’s *Build a Large Language Model (From Scratch)* [3] for the model-side concepts (transformer internals, position encodings, prompting, and how to think about explainability), and the *Mémento de tactique appliquée* of the École de Guerre-Terre [4] for the order templates, tactical factors, and doctrinal constraints. The brief framed latency as a per-token figure, but the measure that actually matters to an operator is wall-clock time per SITAC; that is what I tracked, and Section 4.2 reports it.

Table 4: Principal tools and methods, and their role in the system.

Tool / method	Role
Python + typed schemas	Implementation language; typed contracts at every stage boundary.
Local LLM runtime	The generative model, kept local for confidentiality and reproducibility.
GOAP ontology	Formal backbone defining a legal action decomposition.
CPM / CP-SAT solver	Deterministic scheduling, with an optional constraint solver behind a safe fallback.
Streamlit	Menu-driven operator interface with a mandatory human gate.
Raschka [3] / Mémento [4]	Model-side and doctrine-side reference works.

2.5 Regulatory aspects, quality and risk management

In a critical context, the governing risk is not a crash but *silent incorrectness*: an output that is wrong yet reads exactly as confidently as one that is right. The project therefore treats determinism, traceability, and the marking of machine-generated fields as primary quality controls rather than as nice-to-haves. A tool that fails loudly is recoverable; one that fails quietly, by emitting a plausible but doctrinally invalid or physically impossible plan, is the real hazard. Quality engineering was oriented at making failures *detectable* rather than at making the model more capable. Table 5 sets out the main risks and the architectural answer to each.

Risk management here is mostly architecture, built into the stage boundaries rather than bolted on at the end. The recurring pattern is the same: the control is a property of how the modules are wired, so it cannot be forgotten or switched off in the course of normal use.

2.6 The challenges identified at the outset

The design phase also produced an honest list of the hard problems the project would face, and it is worth recording them because they shaped what was attempted and what was deferred. Three stood out.

Table 5: Risk register: principal risks and mitigations.

Risk	Mitigation built into the architecture
Silent incorrectness	Independent rule engine that checks output without trusting its source; failures surfaced as typed findings.
Hallucinated facts	Bounded model surface; physical-feasibility and doctrinal checks downstream.
Non-reproducibility	Content-derived caching; same input yields byte-identical output.
Automation bias	Mandatory human gate; provenance marking of machine-generated fields.
Confidentiality	Local model; no external calls; no fine-tuning on sensitive data.

The first is *latency*. A decision aid that is meant to support a commander in near real time cannot take minutes to produce an option. The brief expressed this as a per-token target, but the operationally meaningful figure is wall-clock time per SITAC. Keeping the model local and its surface small was the design response, and the per-SITAC figures are encouraging: the fully deterministic pipeline completes in roughly a second per SITAC, and the LLM-enhanced (brief-assisted) mode in around ten seconds, both comfortably inside what near-real-time decision support needs.

The second is *integrating real-time data*. A real command picture is fed by geolocation, sensor streams, and after-action feedback (RETEX), and a fielded generator would have to consume that flow rather than a static order. The prototype works from a document rather than a live feed, which is the right scope for a research prototype but is a real distance from an operational one, and bridging it is part of what a downstream integration, for instance with an embedded or simulation environment such as HexaForce, would entail.

The third is *doctrinal and ethical compliance*. The output must respect not only French national doctrine but the broader allied framework and, above all, the legal constraints on the use of force. Encoding doctrine as machine-checkable rules addresses the first; the ethical constraints, examined in Section 4.4, are addressed by keeping a human in genuine authority rather than by anything the machine decides on its own. Naming these challenges at the start kept the project honest about the difference between a research prototype and a system anyone would field.

The fourth challenge is usability: will the system ever reach a level of output quality that justifies operational deployment, or will it require a fundamentally different architecture to do so? The results of the pipeline, while encouraging and steadily improving, are not yet at a level to be used in actual combat, and there is little to no way to know if and when this build will reach a stage where it is usable, or will instead need a new architecture, methods, and tools to ever be deployed.

3 Progress of the internship

Over the internship, the system grew from a parser into a full deterministic-first chain, plus an opt-in simulation that stress-tests a plan against a reactive adversary. This section follows that growth in a deliberately non-technical register: the missions, the way the architecture enforces its own principles, a walk through the pipeline, the concrete pieces I built, the doctrinal context, and how I ran a solo project end to end.

3.1 The missions

My missions covered work across the chain, and each piece landed behind a written gate before it was enabled. The missions were not evenly sized. Some were stage builds; some were guardrails that cut across stages, such as a feasibility check that rejects physically impossible movements and the reproducibility work that lets two runs produce identical output. The simulation subsystem was an opt-in addition built late, as a way to *probe* the rest of the pipeline rather than to extend it. The common thread is that almost every mission was a concrete answer to one question: how do we keep an automated planner trustworthy?

3.2 The two earlier builds and what they taught

The system described here is the third build, and the two that preceded it are part of the story because each failed in an instructive way. The first build was model-led: I leaned on the LLM to do most of the work, generating plans more or less end to end and checking them lightly afterward. It produced fluent output and was quick to demonstrate, but it was exactly the architecture the rest of this report argues against: when it was wrong, it was confidently wrong, and the checking was too thin to catch it. The lesson was that putting the model at the top of the stack makes a good demo and an untrustworthy system.

The second build over-corrected. It pushed almost everything into deterministic code and bolted structure on as the project went, without a clean contract between stages. It was more trustworthy but had become tangled: responsibilities blurred across modules, the model could be reached from several places, and changing one stage risked breaking another in ways I could not always foresee alone. The lesson there was that determinism is necessary but not sufficient; without clear, enforced boundaries a deterministic system is just a different kind of unmaintainable.

The third build, the one this report describes, is the synthesis of those two lessons: determinism-first, but with the boundaries made structural so that the principle cannot quietly erode, and with the model reachable through exactly one client so its influence is auditable.

In hindsight it was right, because each rebuild revealed weaknesses and strengths I could carry into the next iteration, and those properties became the backbone of the final system.

3.3 How the architecture enforces its own principles

The architecture enforces its principles structurally, not by convention, and that is the methodological heart of the project. Figure 1 shows the chain. A handful of boundaries do most of the work. Only one stage reads the raw input, so the rest of the pipeline only ever sees typed structure. Only one client may reach the model, so the model's reach is a single auditable boundary rather than something scattered across the code. The component that builds the operator briefing cannot see the original tactical picture, so it can only present the validated result. And each doctrinal concept has exactly one definition, so nothing can drift out of sync. Because these boundaries are structural, each principle is a property of how the modules are wired rather than a rule someone must remember.

3.4 A walk through the pipeline

It helps to follow a single order through the chain. The **parser** reads the order's prose and produces a typed, read-only tactical picture: it first sorts the document into doctrinal sections by deterministic detection, and only then lets a bounded model step read each section and emit

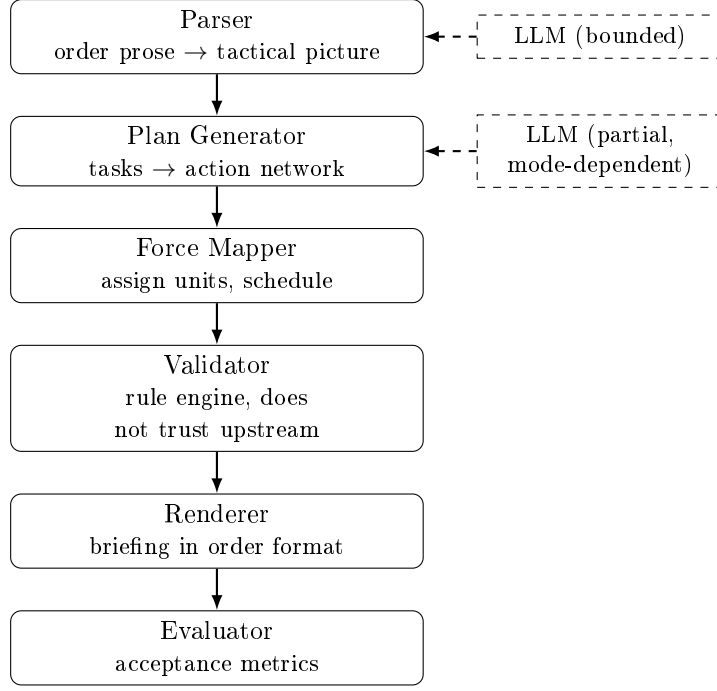


Figure 1: The determinism-first pipeline. The model is reachable only through a single client and touches only two stages; everything load-bearing is deterministic, and the validator and evaluator contain no model at all.

typed entities, which are checked against a schema before assembly. The model’s job is narrow: prose in, structure out, and it never decides what the pipeline does next.

The **plan generator** turns the tasks in that picture into a network of actions. As described below, it can do this in more than one mode, but in every mode the result is a structure the ontology recognises as legal. The **force mapper** then assigns concrete units to those actions and schedules them, producing a timeline with a makespan. The **validator** checks the whole result against doctrine and physics without trusting any upstream stage. If it finds a problem — a unit asked to be in two places at once, a movement that cannot be made in the time allowed, a force component used out of doctrinal order — it raises a typed finding rather than silently passing the plan on. The **renderer** turns the validated plan into the five-paragraph briefing an officer reads, and, crucially, it cannot see the original tactical picture, so it can only present what survived validation. Finally, the **evaluator** computes acceptance metrics, makespan, idle time, and the like, without the model. At every step, the human can stop, inspect, and override; nothing advances on its own.

3.5 Personal achievements

My contributions span the whole chain. What follows highlights the pieces I am most responsible for.

Building the plan. The plan generator offers more than one mode. The default, deterministic mode decomposes the task using the GOAP ontology as its grammar, so it cannot produce an illegal structure. An alternative, model-first mode inverts the order: the model proposes a candidate plan and the rule-based layer then validates and repairs it against the ontology, so the model’s draft is checked before it is trusted rather than emitted directly. A third, enrichment mode lets the model fill bounded descriptive fields inside an already-validated structure. The modes differ in how much of the first draft the model contributes; they do not differ in whether

the result is verified. The piece I am most pleased with here is a concurrency model that lets continuous activities such as surveillance and fire support run alongside other actions instead of blocking them in a queue, which previously inflated idle time for no doctrinal reason.

Assigning and scheduling forces. The force mapper uses a greedy assignment and a deterministic scheduler as the baseline, with an optional constraint solver on top behind a “first do no harm” fallback: if the solver is unavailable, infeasible, or simply no better, the system keeps the baseline schedule unchanged. The solver can only improve the plan, never break it.

Checking the result. The validator is the rule engine, and it does not trust its upstream, the model included. The feasibility check is the piece that best captures the project’s spirit: it resolves the named origin and destination of a movement to coordinates, computes the distance, and rejects the move when a unit could not physically cover that ground in the time the plan allows. It catches a class of error that content-level checking misses, because the model knew the place names but not the distances between them.

Probing the plan with a simulated adversary. A simulation subsystem, added late, runs the same planner against a reactive adversary and adjudicates engagements with a deterministic judge that contains no model at all. The deliberate absence of a model in the judge is the point: the part of the system that decides whether something is acceptable does not itself depend on a probabilistic component. The loop shows how a plan adapts turn by turn under pressure, and it is the closest the project comes to a robustness measurement.

Making it reproducible. Cutting across all of this is the reproducibility work: replacing a fragile cache key that never matched across runs with a stable, content-derived one, so the same input yields byte-identical output despite the random identifiers generated along the way. It is the least glamorous thing I built and, I think, the most important, because without it there is nothing stable to measure.

3.6 Work in a doctrinal context

Working against French doctrine meant encoding national tactical terminology and ordering rules as machine-checkable constraints rather than carrying them as free text. The eight action modes, the philosophies attached to them, the ordering of force components, and the matching of a unit’s level to the scale of its task are not prose the model is asked to honour; they live in data and are enforced by the validator. Much of the doctrinal effort was a translation exercise: turning doctrine written for human officers, drawn from the *Mémento de tactique appliquée* [4], into a form a deterministic rule engine can check, so that compliance is a property the system can demonstrate rather than a claim it makes. The periodic checks with officers were where I learned that a plan can be schema-valid and still tactically odd, which is precisely why the human gate is not optional. A rule engine can verify that a plan breaks no encoded rule; it cannot verify that the plan is wise, and the gap between “valid” and “wise” is exactly the space the human occupies.

3.7 Probing robustness: the simulated adversary

The simulation subsystem deserves its own description because it is the closest the project comes to measuring robustness rather than asserting it. The idea is simple: a plan that looks good in isolation may fall apart against an opponent who reacts, so the system runs the same planner a second time on a mirrored picture to produce an adversary’s plan, then plays the two against each other turn by turn. Each engagement is adjudicated by a deterministic judge that maps the local balance of forces to a discrete outcome, and a per-turn difference record shows how the

friendly plan adapts as the situation changes.

Two design choices matter for the report’s theme. First, the judge contains no model at all: the part of the system that decides who prevails in an engagement is plain, reproducible arithmetic, so the adjudication cannot hallucinate. Second, the loop reuses the same planner for both sides rather than building a separate adversary, which keeps the experiment honest and cheap. What the loop gives is a qualitative picture of whether a plan degrades gracefully or collapses under pressure; what it does not yet give is a systematic catalogue of failure modes or a controlled comparison of the planner with and without each guardrail. That catalogue is the natural next experiment, and I record it as designed but unrun rather than claiming a robustness result the data does not yet support.

3.8 Project ownership and self-imposed process

There was no team to coordinate, so “management” meant managing myself and the work. The substance of it was process discipline, because the project’s value rests on being auditable and reproducible, and that is hard to keep honest when you are the only person on it. Changes moved through a fixed set of steps before landing — settling the architectural call, implementing it, checking it against its gate, and writing it up — and I kept a documentation trail current as I went so the rationale for a decision survived past the day I made it.

To make the workflow tractable alone I set up and drove a set of AI tooling agents — an architecture advisor, a code writer, a reviewer, and a test engineer — each with a defined remit. Those are tools I built and directed, not colleagues; the judgement and the responsibility for what landed were mine, and I treated their output as input to a decision rather than as a decision. The arrangement is itself a small instance of the report’s theme: a useful but unreliable assistant, kept on a short leash, with a human making the call. On a solo project, that written record and that disciplined process are the only memory and the only review the project has beyond my own.

4 Internship analysis

***Key topic.** The internship sharpened one engineering judgement above the others: knowing where a model may safely act in a pipeline is worth more than making the model more capable.*

4.1 Skills developed, acquired and missing

The internship deepened a cluster of design skills around bounding a model safely and around evaluation, and it exposed an equally specific set I still need to develop.

The skill that grew most is contract-driven, typed-pipeline design. Before this project, I treated schemas as a way to catch bugs; here they became the load-bearing structure of the whole system. Putting a typed contract on every stage boundary, so that a stage physically cannot hand the next a malformed artefact, changes how you reason about the code: you stop re-checking that data is well-formed in the consuming function because the type already guarantees it. Designing those boundaries so that the determinism-first principle becomes a property of the wiring, rather than a rule I have to remember, was the single most useful thing I learned to do.

Close behind is the discipline of treating doctrine as data with one source of truth, enforced by a rule engine that does not trust its upstream, which transfers well beyond the military domain.

I also came out of the project thinking differently about LLMs: I began thinking about prompts and finished thinking about where in a pipeline a model may act, what bounds its output, and how its failures are made detectable. Reading the trustworthy-LLM literature for Part II, and the model-internals chapters of Raschka [3], turned that reading into evaluation thinking rather than survey knowledge, and gave me a working mental model of why architectural choices in a model matter and not only how to prompt one. On the operations-research side, I built working familiarity with scheduling and constraint solving that I had only met in coursework before.

A skill I did not expect to develop, and the one I am least able to put on a CV, is solo project discipline: owning an architecture end to end with no co-developer, and substituting written gates and a documentation trail for the reviewer I did not have.

The gaps are as specific as the gains. Formal verification is the clearest: my rule-following guarantee rests on a deterministic validator and a schema, which is sound engineering, but I cannot prove properties of the pipeline the way a formal-methods toolkit would. Constraint-solver tuning is another: I can wire a solver behind a safe fallback, but I am not yet fluent at shaping the model or the search so it reliably beats the baseline rather than merely matching it. The third is larger-scale empirical evaluation, which is a methodological skill I have only begun to build, and which is why the evaluation phase is the thinnest part of the project. The honest summary is that I am a stronger systems designer than I was, and that the gaps are now precise enough to be a plan rather than a worry.

4.2 Difficulties and solutions

The recurring difficulty was reconciling probabilistic generation with hard guarantees; the answer was architectural rather than a matter of prompt-tuning. Working solo and a troubling efficiency result were the others.

The difficulty that ran through the whole internship was the obvious one in hindsight: a model is a probability distribution, doctrine is a set of hard constraints, and the two do not reconcile by being asked nicely. My first instinct was to treat it as a prompting problem, and that did not hold. The solution I converged on is the spine of the report: rather than trusting the model to follow rules, I confined it, so it operates inside a structure already validated, fills only bounded fields, and passes everything through a rule engine that does not trust it.

The second difficulty was working alone, with no one to catch the design mistake I could not see, and on a system whose entire value rests on being auditable, that is a real hazard rather than a minor inconvenience. I could not manufacture a second engineer, so I manufactured the next best things: written gates and periodic checks with officers for the doctrinal side. It is an imperfect substitute, since a gate I wrote cannot question the assumptions I wrote it under, but it imposed the discipline a reviewer would have.

The third is a result I want to state plainly rather than dress up. A sweep of the planner's schedules showed roughly sixty percent of unit time sitting idle. That is too high for a plan anyone would want to field, and I record it here as a problem, not a success. Some of that idle time is probably legitimate waiting inherent in tactical movement, but I could not establish how much, and treating the whole figure as benign would be exactly the comfortable rationalisation a trustworthy-systems discipline is supposed to resist. The honest position is that the number is a real weakness, its breakdown into benign and wasteful idleness is unproven, and reducing it is open work. The concurrency model I added was a first step against it, but I have no clean before-and-after measurement to claim a specific improvement, so I do not claim one.

Latency, by contrast, turned out better than feared once it was measured in the right unit. The

brief expressed it per token, but per-SITAC wall-clock is what an operator feels, and by that measure the system is comfortably fast: a fully deterministic run completes in about a second per SITAC, and the LLM-enhanced (brief-assisted) mode in around ten seconds, the difference being the cost of the model calls the deterministic path avoids. The honest caveat is that this is wall-clock on a development machine rather than a profiled, per-component budget on target hardware, so it is a reassuring figure rather than a fielded guarantee.

4.3 Successes, best practices, areas for improvement

The clearest successes are the ones the system can demonstrate about itself: a reproducible end-to-end run, a rule engine that catches doctrinal and physical errors, a genuinely small model surface. They sit beside honest gaps that are mostly about unfinished measurement, and one result that is simply a problem.

The success I am most confident in is that the system runs, reproducibly, end to end, on real orders, and that I can show it. Reproducibility is the property that makes any later measurement mean anything. The second success is the validator as an independent rule engine that catches both doctrinal and physical inconsistencies, the feasibility check most of all. The third is the bounded-model design itself: of the places that could call the model, only a few do, and the parts that judge acceptability contain no model at all.

Three best practices emerged that I would carry to any critical-systems project, and they are general enough to outlive this one: gated changes, where nothing lands until it passes a written criterion; determinism-first, where the model is used only where deterministic logic cannot reach; and single-source-of-truth data, so nothing drifts.

The areas for improvement are forward-looking and mostly about evidence. Table 6 sets out the evaluation dimensions the brief called for and their honest status. The evaluation phase is the thinnest part of the project: the ablation that would isolate each guardrail’s contribution, the feasibility check’s accuracy against human judgement, and the idle-ratio decomposition and reduction are all designed but unrun. The system has also never been put in front of a serving planning officer for a genuine operator study, which is the only test that would validate the human-in-the-loop claims. Against the initial objectives — mine to learn how to make a model acceptable in a critical context, and the company’s to produce a reproducible, auditable, doctrine-checked prototype — I judge the architectural objectives met and the empirical-evaluation objectives only partly met and clearly scoped for whoever continues the work.

4.4 Critical analysis, ethics, and DD-RSE commitments

Responsible AI in defence rests on auditability and human authority, and the value of this architecture is that it makes both structural rather than optional. Determinism-first carries its own costs and residual risks, which a fair analysis has to name.

Stepping back, the internship is a worked example of an ethical position as much as a technical one. When a tool can influence how force is applied, trust is not a feature; it is the precondition for the tool existing at all, and the two commitments that matter are that machine reasoning be auditable and that human authority be preserved. Those are exactly the commitments the company’s published posture names, and the architecture’s contribution is to make them structural rather than discretionary. Auditability is not a logging feature switched on at the end; it falls out of determinism and provenance, because every machine-touched field is marked and the same input reproduces the same output. Human authority is not a usage policy; it is an interface

Table 6: Evaluation dimensions from the brief and their honest status.

Dimension	Intended measure	Status
Efficiency	Makespan, idle ratio, decision-time reduction	Measured; idle ratio is too high (a problem)
Reliability	Run-to-run reproducibility	Demonstrated
Rule-following	Validator pass / violation rates	Runs by default; accuracy vs. human labels unrun
Robustness	Behaviour under adversarial pressure	Simulation runs; systematic study unrun
Latency	Wall-clock time per SITAC	~ 1 s/SITAC deterministic, ~ 10 s with LLM enrichment (dev-machine wall-clock)
Human factors	Operator study with officers	Study not yet conducted; system not yet at deployment readiness

that never auto-advances and a rule engine that signs nothing off on its own. A commitment the system cannot operate without is worth more than one an operator can switch off.

An analysis that only praised the design would not be worth much. Determinism-first has a ceiling: it makes the model safe by making it small, so the system inherits every coverage gap in its deterministic backbone, and where doctrine is genuinely ambiguous it has no answer of its own and must defer to the human. The human-in-the-loop is itself a cost as well as a safeguard. A mandatory review at every stage adds latency and cognitive load, and a tool that asks for sign-off too often invites the rubber-stamping that automation bias predicts, which would quietly defeat the very oversight it was meant to guarantee. The right design directs the operator’s attention where risk concentrates rather than demanding uniform approval, and that proportional design is something my prototype gestures at but does not yet achieve.

There is an ethical point underneath all of this that no architecture settles. A fluent recommendation lends unearned authority to a decision: it looks finished, it reads confidently, and it is easy to accept without genuine examination. The harder a system makes it to act without actually understanding the recommendation, the better it serves the human whose responsibility the decision ultimately is. That is why explainability and provenance are not cosmetic in this domain; they are the mechanism by which a human’s sign-off can be made meaningful rather than nominal. My recommendation, were the prototype to go further, follows from these limits: invest in the measurement that would prove the guardrails work, design the human gate to ask for attention proportionally, and treat the coverage of the deterministic backbone as the real frontier, because that, not the model, is where the system’s competence stops.

4.5 Recommendations for continued work

Because the prototype is meant to be picked up by someone else, it is worth setting out, concretely, what I would do next. The recommendations fall into three groups, and they follow directly from the gaps named above rather than from a wish-list.

Finish the measurement. The single most valuable next step is to run the experiments that are already scaffolded: an ablation that isolates what each guardrail contributes, an accuracy study of the feasibility check against human judgement, and a proper decomposition and reduction of the idle ratio. Until these are done, the project’s central claims rest on argument rather than

evidence, and the prototype’s whole point is to replace assertion with measurement.

Close the distance to an operational setting. Two steps matter here: an operator study with serving officers, which is the only way to test the human-in-the-loop claims that no internal test can reach, and a first integration with a live or simulated data feed rather than a static document, which is where the real-time-data challenge of Section 2 actually bites.

Harden the design where it is thin. The deterministic backbone’s coverage, not the model, is where the system’s competence stops, so extending and stress-testing that coverage is the real frontier. The human gate should be made proportional, asking for scrutiny where risk concentrates rather than uniformly, so that it resists the rubber-stamping automation bias predicts. And the model-first generation mode deserves the same rigorous validation the deterministic mode already has, since it is the path most exposed to the model’s errors.

None of these requires a more capable model. All of them are about turning a defensible architecture into a measured, integrated, and hardened one, which is the work that stands between a research prototype and something that could be trusted in the field.

4.6 Career plan

The internship pointed me toward trustworthy-AI and critical-systems engineering, building the structure around models rather than the models themselves, and that is the direction my short- and medium-term plans follow.

What this internship clarified for me is less a job title than a kind of problem I want to keep working on: making powerful but unreliable components safe to deploy where being wrong has real consequences. In the short term I want to consolidate the skills Section 4 flagged as thin — formal reasoning about system properties, constraint-solver fluency, and rigorous empirical evaluation — because those are the gaps between the architecture I can build today and the proofs a fielded critical system actually demands. A first role in trustworthy-AI, critical-systems, or decision-support engineering, where the hard part is the verification and not the demo, is what I am aiming for.

In the medium term, I would like to grow from implementing these architectures to designing them at a larger scale: the engineer who decides where a model may safely act across a whole system, not just within one pipeline, and who can defend that boundary to both a customer and a regulator. The combination this internship gave me — a defence and responsible-AI context, a determinism-first design instinct, and a habit of measuring trust rather than asserting it — is uncommon, and I want to build a career on it.

References

- [1] Thales Group, “The Thales TrUE AI approach to Artificial Intelligence to be unveiled at the Paris Air Show 2019,” press release, June 2019. [thalesgroup.com](https://www.thalesgroup.com).
- [2] Thales Group, *Digital Ethics Charter*, [thalesgroup.com](https://www.thalesgroup.com) (corporate responsibility / digital responsibility).
- [3] S. Raschka, *Build a Large Language Model (From Scratch)*. Shelter Island, NY: Manning, 2024.
- [4] École de Guerre-Terre, *Mémento de tactique appliquée*, French Army doctrine reference.

The scientific references cited in Part II accompany the scientific chapter (the IEEE conference paper).